

BCSE303L

OPERATING SYSTEMS

Dr.N.Narayanan Prasanth

Professor

Department of Database Systems

School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Mail: **n.prasanth@vit.ac.in**

Cabin: SJT116 A40

Objectives

1. To introduce the operating system concepts, designs and provide skills required to implement the services.
2. To describe the trade-offs between conflicting objectives in large scale system design.
3. To develop the knowledge for application of the various design issues and services.

Course Outcomes

On completion of this course, student should be able to:

1. Interpret the evolution of OS functionality, structures, layers and apply various types of system calls of various process states.
2. Design scheduling algorithms to compute and compare various scheduling criteria.
3. Apply and analyze communication between inter process and synchronization techniques.
4. Implement page replacement algorithms, memory management problems and segmentation.
5. Differentiate the file systems for applying different allocation, access technique, representing virtualization and providing protection and security to OS.

SYLLABUS

Module:1	Introduction	3 hours
Introduction to OS: Functionality of OS - OS design issues - Structuring methods (monolithic, layered, modular, micro-kernel models) - Abstractions, processes, resources - Influence of security, networking, and multimedia.		
Module:2	OS Principles	4 hours
System calls, System/Application Call Interface – Protection: User/Kernel modes - Interrupts -Processes - Structures (Process Control Block, Ready List etc.), Process creation, management in Unix – Threads: User level, kernel level threads and thread models.		
Module:3	Scheduling	9 hours
Processes Scheduling - CPU Scheduling: Pre-emptive, non-pre-emptive - Multiprocessor scheduling – Deadlocks - Resource allocation and management - Deadlock handling mechanisms: prevention, avoidance, detection, recovery.		
Module:4	Concurrency	8 hours
Inter-process communication, Synchronization - Implementing synchronization primitives (Peterson's solution, Bakery algorithm, synchronization hardware) - Semaphores – Classical synchronization problems, Monitors: Solution to Dining Philosophers problem – IPC in Unix, Multiprocessors and Locking - Scalable Locks - Lock-free coordination.		

SYLLABUS

Module:5	Memory Management	7 hours
Main memory management, Memory allocation strategies, Virtual memory: Hardware support for virtual memory (caching, TLB) – Paging - Segmentation - Demand Paging - Page Faults - Page Replacement -Thrashing - Working Set.		
Module:6	Virtualization and File System Management	6 hours
Virtual Machines - Virtualization (Hardware/Software, Server, Service, Network - Hypervisors - Container virtualization - Cost of virtualization - File system interface (access methods, directory structures) - File system implementation (directory implementation, file allocation methods) - File system recovery - Journaling - Soft updates - Log-structured file system - Distributed file system.		
Module:7	Storage Management, Protection and Security	6 hours
Disk structure and attachment – Disk scheduling algorithms (seek time, rotational latency based)- System threats and security – Policy vs mechanism - Access vs authentication -		

Module 8: Contemporary Issues

2 hours

Text Books

1. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts", 2018, 10th Edition, Wiley, United States.

Reference Books

1. Andrew S. Tanenbaum, "Modern Operating Systems", 2016, 4th Edition, Pearson, United Kingdom.

2. William Stallings, "Operating Systems: Internals and Design Principles", 2018, 9th Edition, Pearson, United Kingdom.

Course Assessment Details

1. Case Study – 10 Marks (Before FAT)
2. Technical Quiz 1 – 10 Marks (Before CAT 1)
3. Technical Quiz 2 – 10 Marks (Before CAT 2)
4. CAT 1 – 15 Marks (will be scheduled by university)
5. CAT 2 – 15 Marks (will be scheduled by university)
6. FAT – 40 Marks (will be scheduled by university)

Module – 1

Introduction to Operating Systems

- ❑ An operating system is system software that manages a computer's hardware.
- ❑ It provides a *platform* for running application software.
- ❑ It is a software that acts as an *interface/intermediary* between computer hardware components and the user.
- ❑ It perform tasks in a wide variety of *computing environments* such as smart phones, personal computers, enterprise computers, home appliances and many more.
- ❑ The OS performs essential tasks such as process management, memory management, file management, device management, and security, ensuring the smooth and efficient operation of the computer system.

Goals of OS

1. To execute user programs and solve user problems easier
2. To make the computer system convenient to use
3. To use the computer hardware in an efficient manner
4. To provide a user-friendly and convenient environment for the user
5. To fully utilize the computing resources

What OS do?

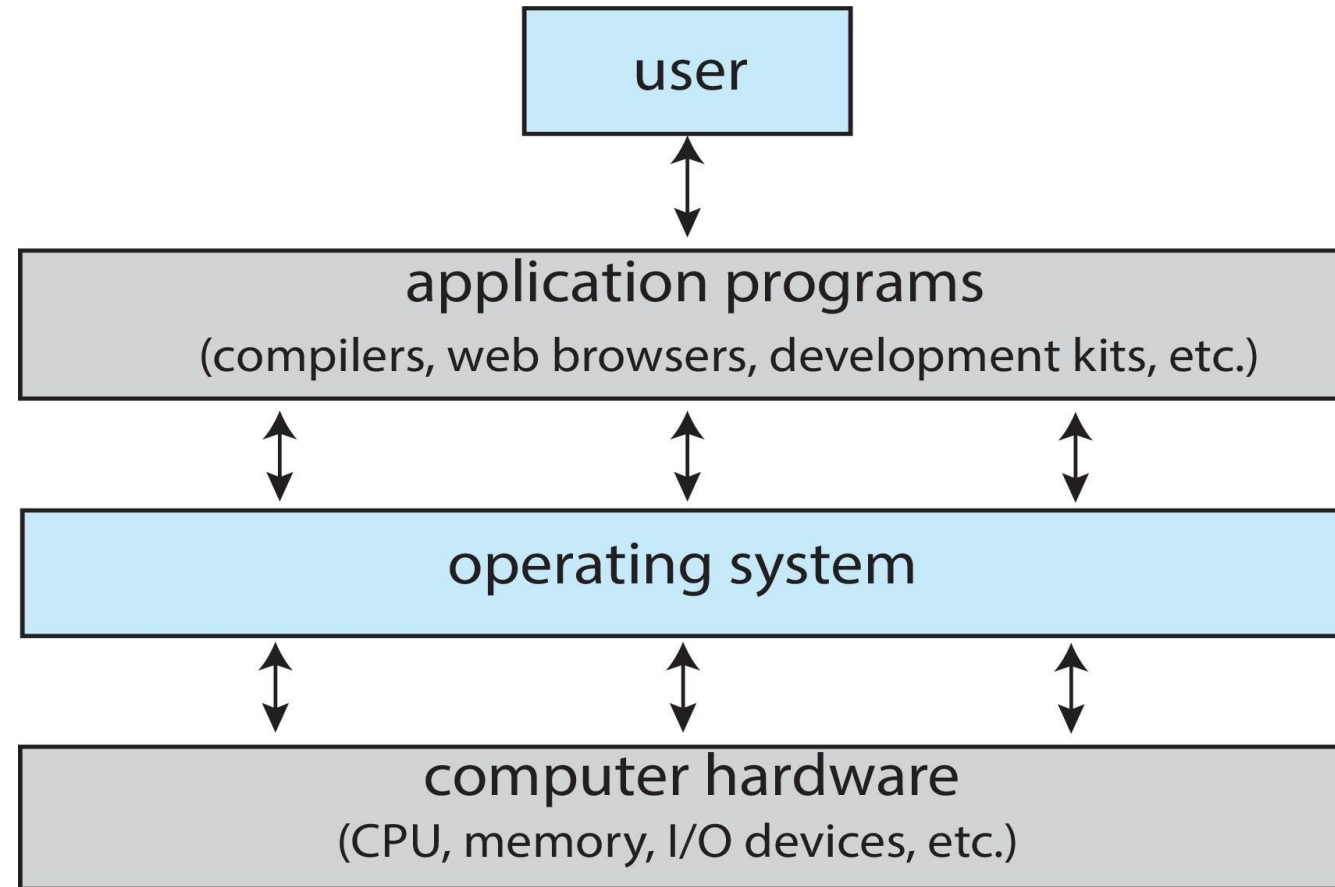


Figure 1 Abstract view of the components of a computer system

OS usage

- Hardware Abstraction
turns hardware into something that applications can use
- Resource Management
manage system's resources

What OS do?

It **controls the hardware** and coordinates its use among the various application programs for the various users.

*Explore operating systems from two viewpoints: **user and system***

User View: The user's view of the computer varies according to the interface being used.

Example: Single user i.e. laptop user and multiple user

(importance given to ease of use, performance and security but not to resource utilization).

What OS do?

System View:

It act as an Resource Allocator

It act as a *manager for many resources* such as CPU time, memory space, storage space, I/O devices, and many more.

It decide *how to allocate* them to specific programs and users so that it can operate the computer system efficiently and fairly.

It emphasizes the need to control the various I/O devices and user programs.

An operating system is a control program

A control program manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

An Example

A Simple Program

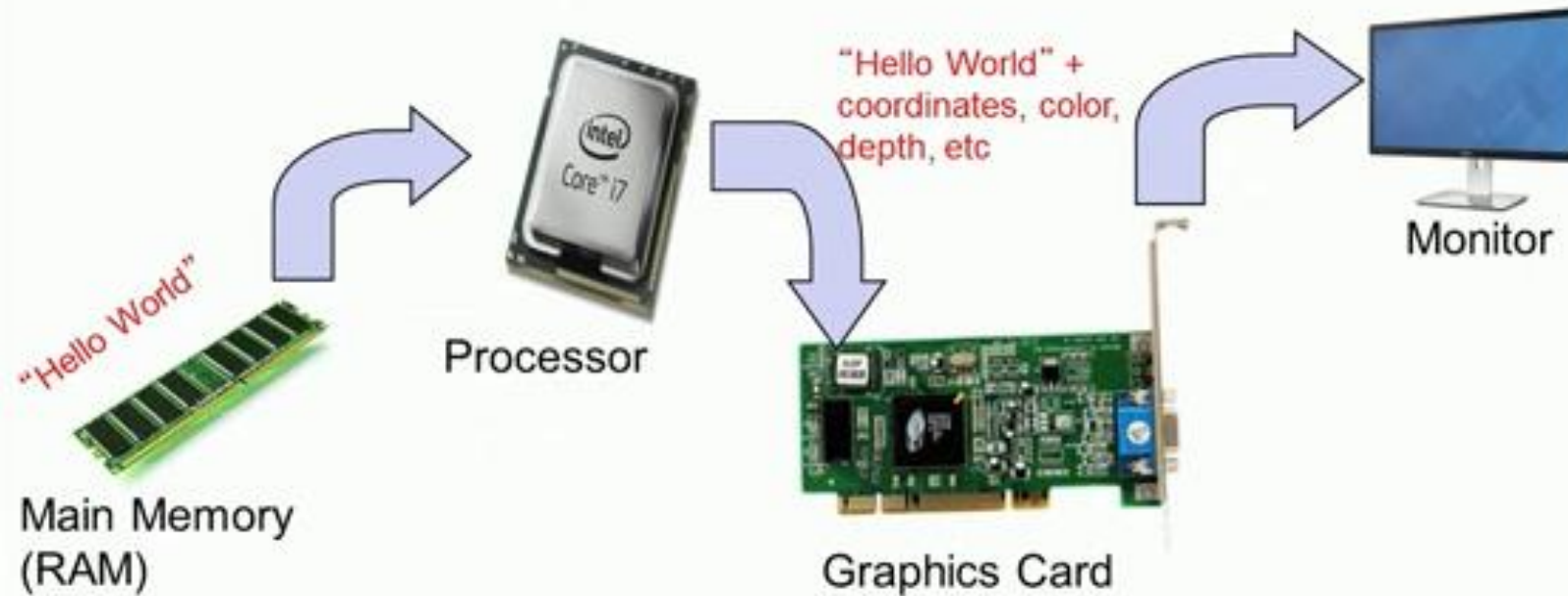
What is the output of the following program?

```
#include <stdio.h>

int main(){
    char str[] = "Hello World\n";
    printf("%s", str);
}
```

How is the string displayed on the screen?

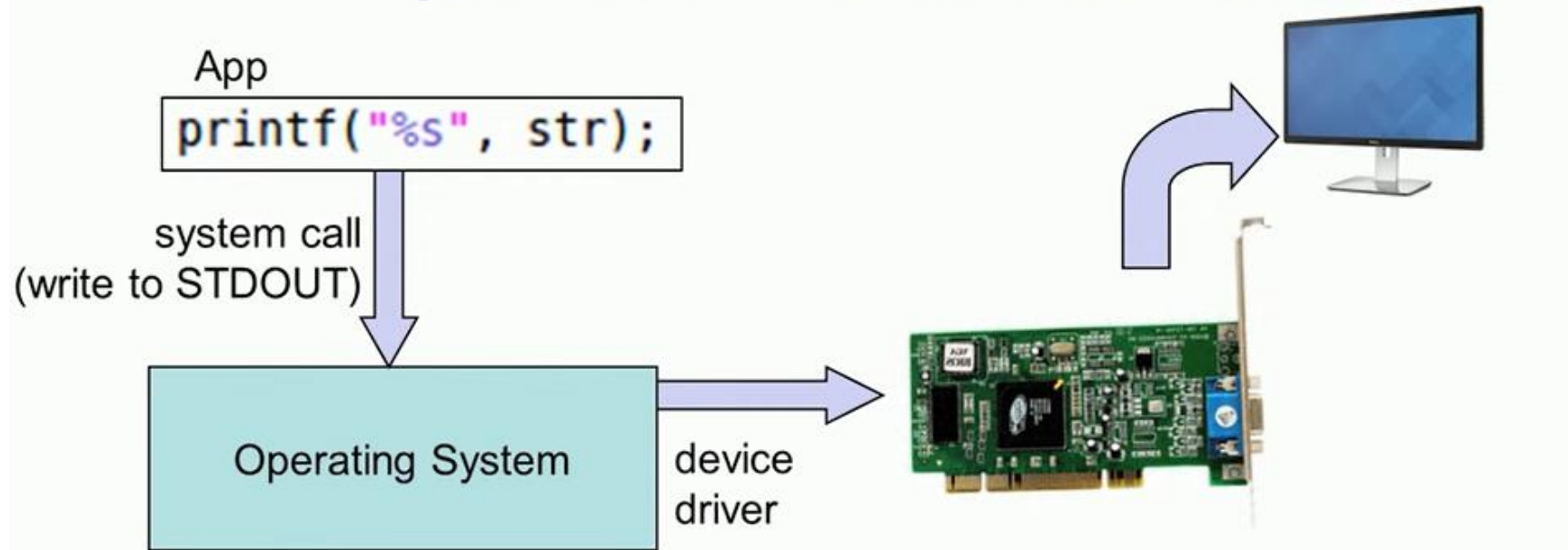
Displaying on the Screen



- Can be complex and tedious
- Hardware dependent

Without an OS, all programs need to take care of every nitty gritty detail

Operating Systems provide **Abstraction**



- **Easy to program apps**
 - No more nitty gritty details for programmers
- **Reusable functionality**
 - Apps can reuse the OS functionality
- **Portable**
 - OS interfaces are consistent. The app does not change when hardware changes

OS as a **Resource Manager**

- Multiple apps but limited hardware

Apps



```
int main() {  
    char str[] = "Hello World";  
    printf("%s", str);  
}
```



Operating Systems



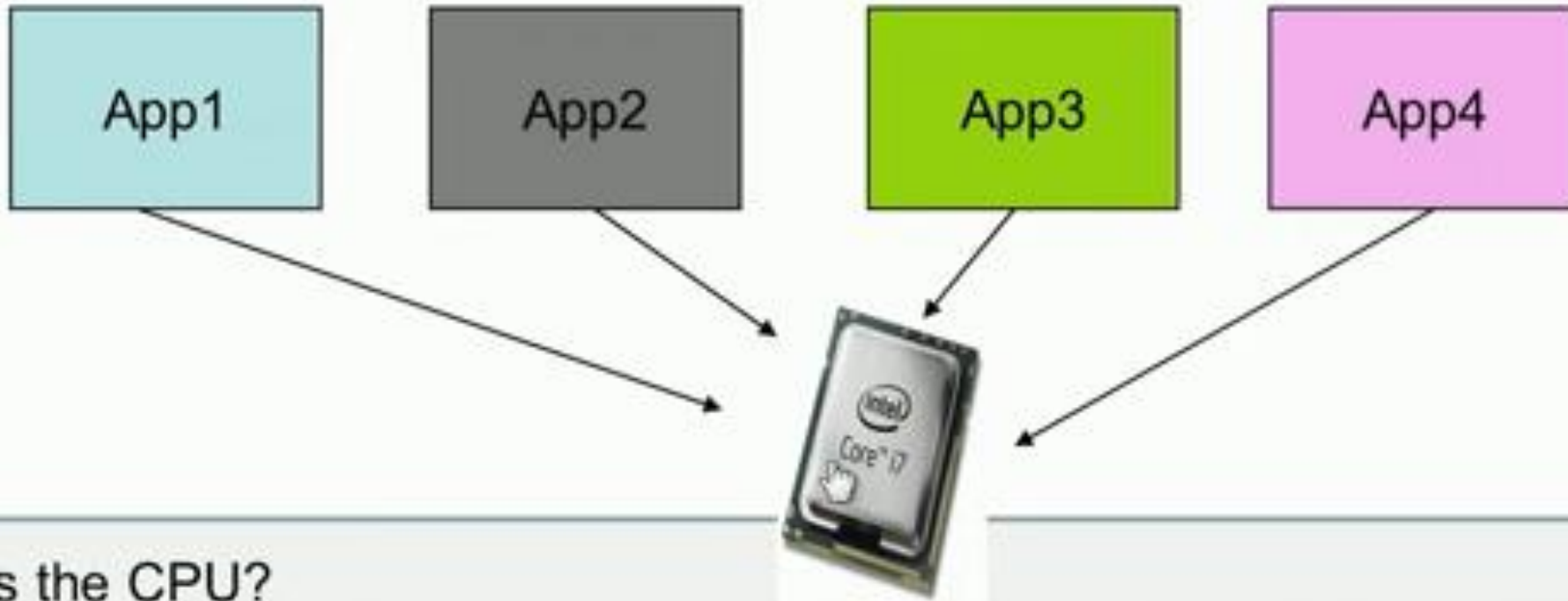
Allows sharing of hardware!!

OS as Resource Manager

- OS must manage CPU, memory, network, secondary storage (hard disk), etc...
- Resource management
 - allows multiple apps to share resources
 - protects apps from each other
 - Improves performance by efficient utilization of resources

For example,

Sharing the CPU



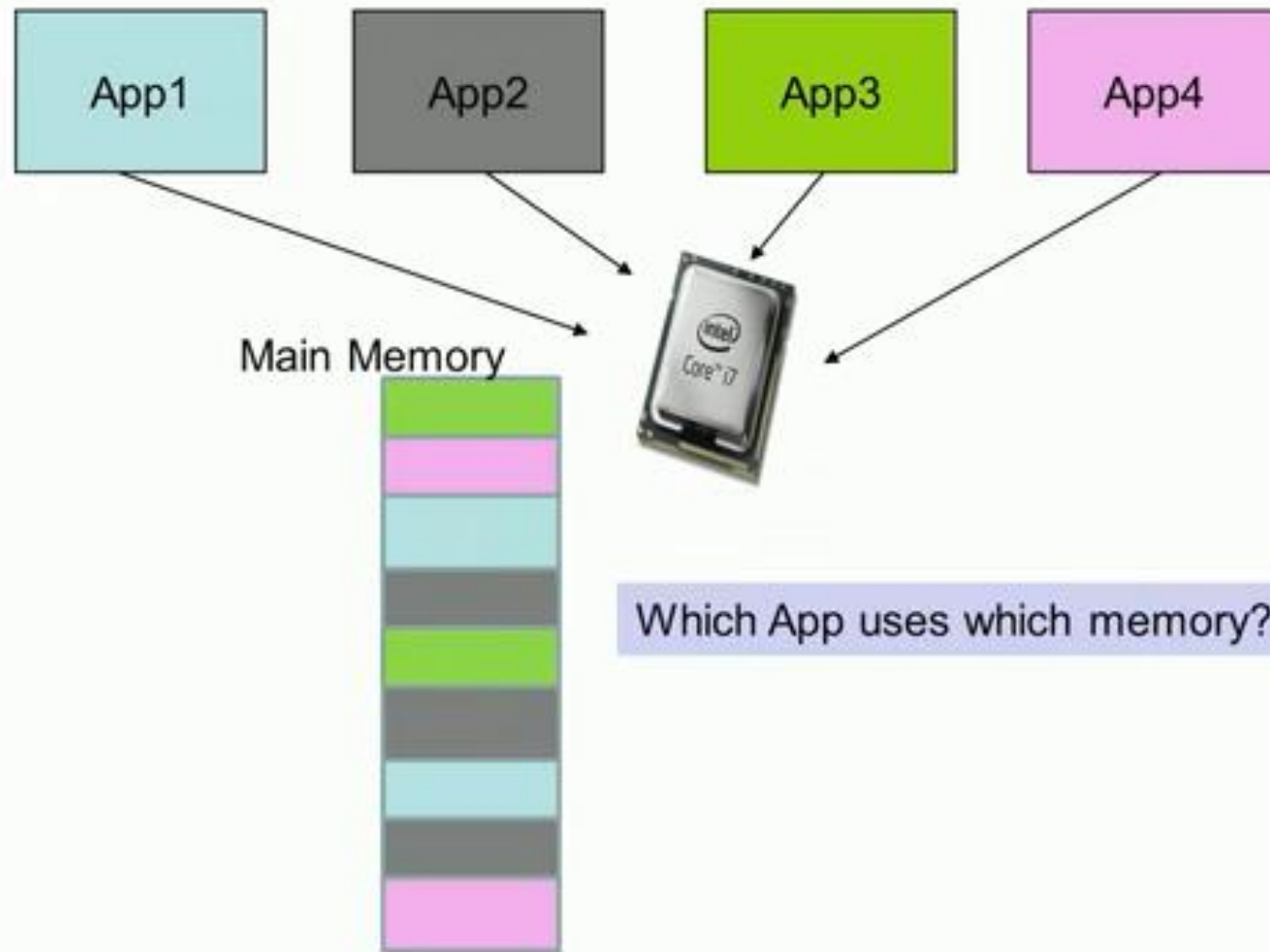
Who uses the CPU?



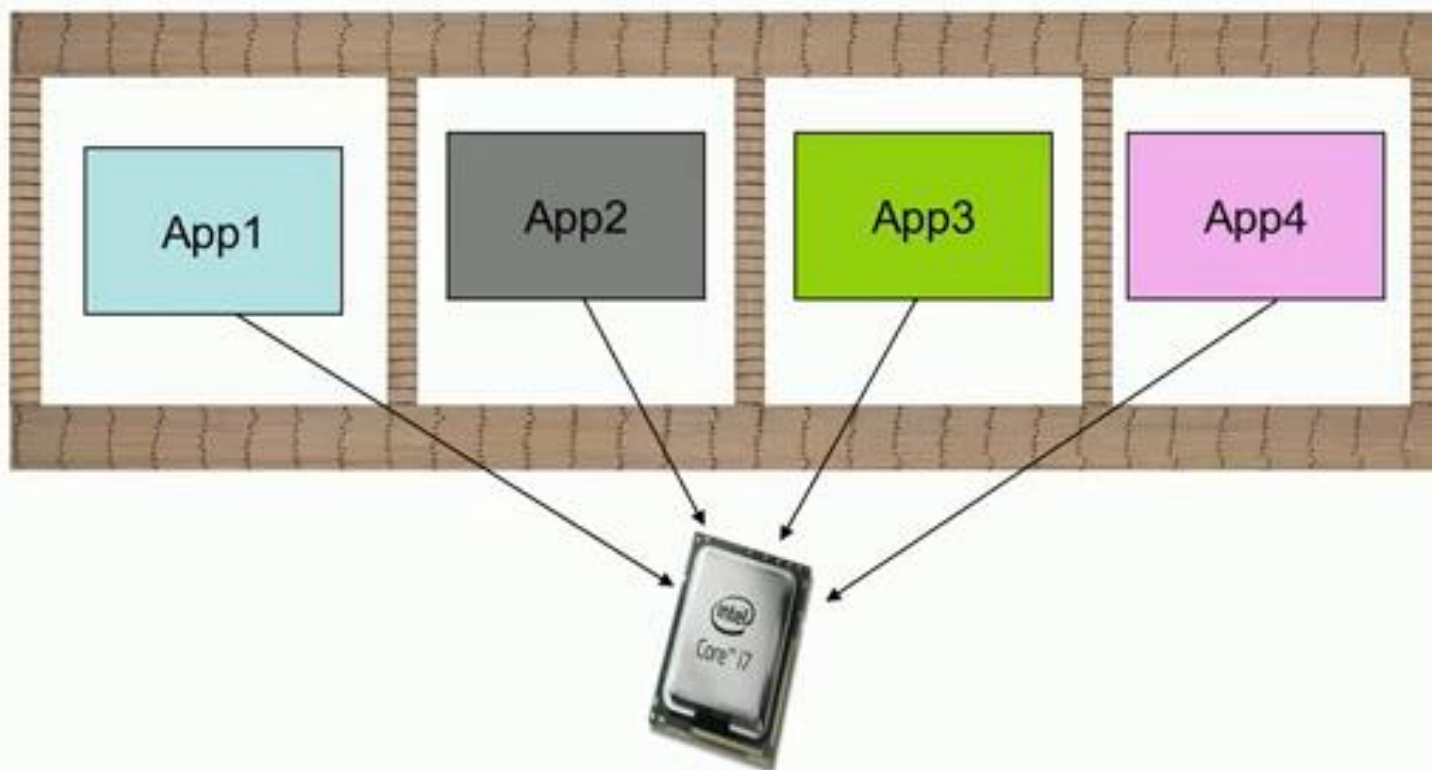
time



Sharing Memory



Share but Isolate



Share resources but keep applications isolated from each other

What OS do?

- *Perform resource allocation.*
- *Control all the programs running in their environment.*
- *Facilitate resource sharing.*
- *Facilitate optimization of programs.*
- *Facilitates different user interfaces.*
- *Makes computer systems to be used easily and with needed performance.*

Resources managed by OS

- Central Processing Unit
- Main memory
- Secondary storage devices
- Input-Output devices
- Network connections
- Peripheral devices

Operating-System Services / Functionalities

- An operating system provides an environment for the execution of programs.
- The specific services provided can differ from one operating system to another.

*One set of operating system services provides functions that are **helpful to the user**.*

1. User Interface: Almost all operating systems have a user interface (UI). They are in different forms. *For example: command-line interface, batch interface, GUI*

Operating-System Services / Functionalities

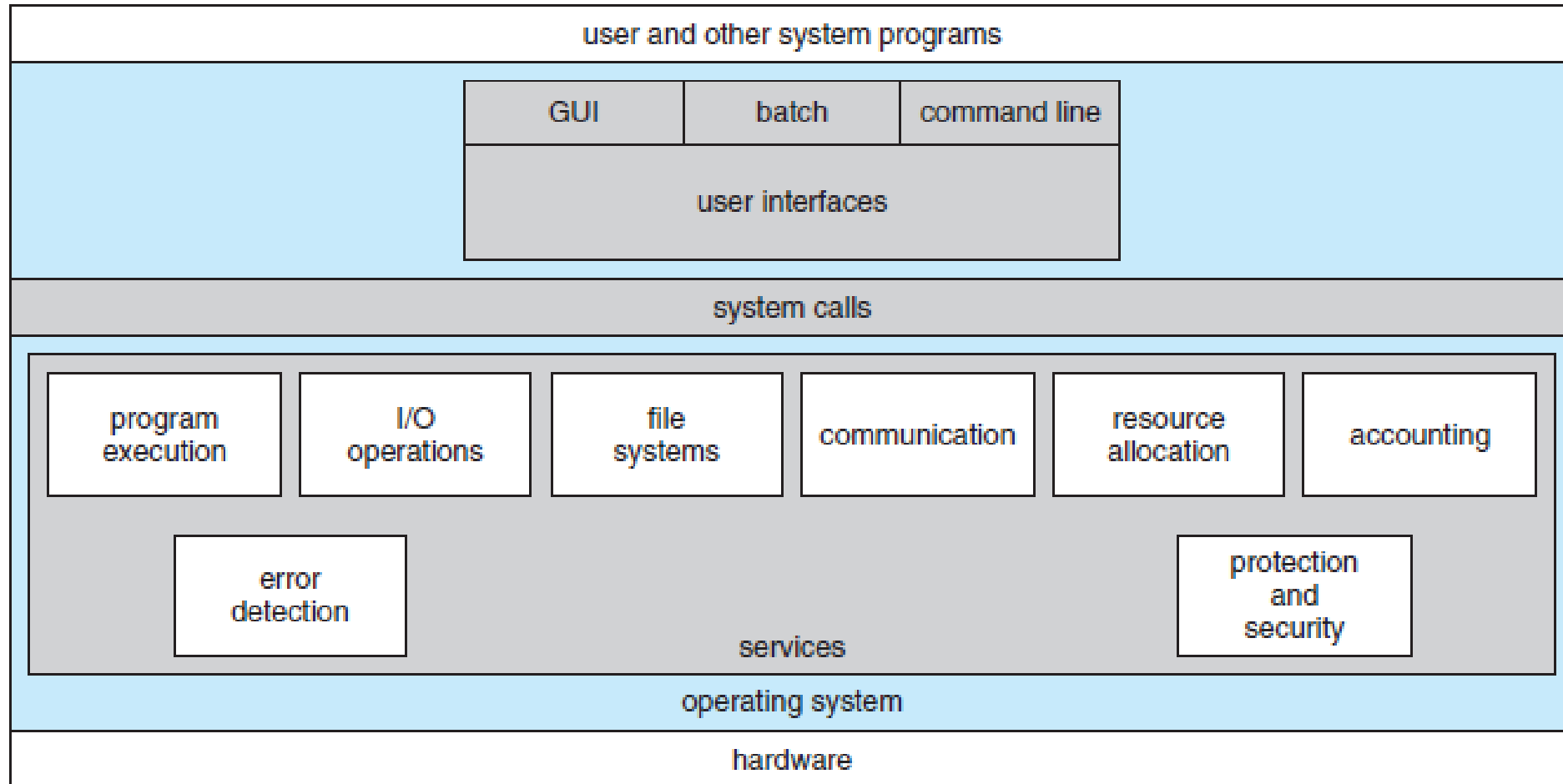


Figure: A view of operating system services

Operating-System Services

- 2. **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally.
- 3. **I/O operations** - A running program may require I/O, which may involve a file or an I/O device.
- 4. **File-system manipulation** - Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
- 5. **Communication** – Process to process communication located on the same or different computer. *Example approaches: Shared memory, message passing*

Operating-System Services

6. Error detection – OS needs to be constantly aware of possible errors.

- *May occur in the CPU and memory hardware, in I/O devices, in user program*
- For each type of error, OS should take the appropriate action to ensure correct and consistent computing.
- Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.

Operating-System Services

Another set of operating system functions exists *not for helping the user but rather for ensuring the efficient operation of the system*. Systems with multiple users can gain efficiency by sharing the computer resources among the users.

1. Resource allocation: When there are multiple jobs running at the same time, resources must be allocated to each of them. Some resources are allocated based on scheduling schemes and some are allocated directly.

Operating-System Services

2. Accounting: Keep track of resource usage details of all the users. This record may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.

3. Protection and security. The owners of information stored in a multiuser or networked computer system may want to control use of that information.

- Protection involves **ensuring that all access to system resources is controlled.**
- Security starts with *user authentication* to gain access to system resources.
- Protect devices from invalid access attempts.

Operating-System Services

Some of the other services are:

- 1. Process management:** The operating system is responsible for starting, stopping, and managing processes and programs. It also controls the scheduling of processes and allocates resources to them.
- 2. Memory management:** The operating system manages the computer's primary memory and provides mechanisms for optimizing memory usage.
- 3. Security:** The operating system provides a secure environment for the user, applications, and data by implementing security policies and mechanisms such as access controls and encryption.

Operating System Services

- 4. File management:** The operating system is responsible for organizing and managing the file system, including the creation, deletion, and manipulation of files and directories.
- 5. Device management:** The operating system manages the communication between the computer system and its peripheral devices such as printers, displays, keyboards, and mice.
- 6. Networking:** The operating system provides networking capabilities that allow the computer system to communicate with other systems and access remote resources.

Operating System Design Issues

The first problem in designing a system is to define goals and specifications.

The design of the system will be affected by the choice of hardware and the type of system: (i.e. traditional desktop/laptop, mobile, distributed, or real time).

The requirements can be divided into two basic groups: user goals and system goals.

User goals - The system should be convenient to use, easy to learn and to use, reliable, safe, and fast.

System goals - the developers ensure the design must be flexible, reliable, error free, and efficient. The requirements vary according to the type of OS designed.

Operating System Design Issues

Separation of Policy and Mechanism

- Mechanism defines how a task is performed.
- Policy defines what should be done or which action should be chosen.
 - Policies are likely to change across places or over time.

Example: The timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision

- A microkernel keeps only the basic functions (i.e. mechanisms) inside the kernel.
- However, Windows, macOS, and iOS combine both mechanisms and policies within the kernel and system libraries

Operating System Design Issues

- Once an operating system is designed, it must be implemented
- Early operating systems were written in assembly language. Now, most are written in higher-level languages such as C or C++.
- The lowest levels of the **kernel** might be written in assembly language and C. **Higher-level routines** might be written in C and C++, and **system libraries** might be written in C++ or even higher-level languages.
- Most Android system libraries are written in C or C++, and its application frameworks which provide the developer interface to the system are written mostly in Java.

Operating System Design Issues

- The only possible disadvantages of implementing an operating system in a higher-level language are reduced speed and increased storage requirements.

References

1. **Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.**
2. **Introduction to Operating System – NPTEL Course.**

BCSE303L

OPERATING SYSTEMS

Module 1 Lecture 2

Key concepts in Operating Systems

1. **Kernel**
2. **Process Scheduler**
3. **Memory Manager**
4. **File System Manager**
5. **User Interface**
6. **Device drivers**

Key concepts in Operating Systems

The **kernel** is the core component of an operating system that manages the system's resources and provides low-level services to other components.

- It manages the operations of memory and CPU time.
- It is responsible for various tasks such as *disk management, task management, and memory management*.

The **process scheduler** is responsible for managing the allocation of CPU time to processes and ensuring that all processes get a fair share of the CPU.

The **memory manager** is responsible for managing the allocation of memory to processes and ensuring that processes do not interfere with each other's memory.

Key concepts in Operating Systems

The **file system manager** is responsible for **organizing and managing the storage and retrieval of data on secondary storage devices** such as hard disks.

- It is a system software responsible for the creation, deletion, modification of the files and managing their access, security and the resources used by them.
- These functions are performed in collaboration with the Device Manager.

Device drivers are software components that provide an interface between the operating system and specific hardware devices such as printers, network cards, and displays.

- It enable OS and other computer programs to access hardware functions without needing to know precise details about the hardware being used.

The **user interface** provides a way for the user to interact with the operating system and perform tasks such as launching programs and managing files.

System Calls

System calls provide an interface to the services made available by an operating system.

These calls are generally available as routines written in C and C++, although certain low-level tasks may have to be written using assembly-language instructions.

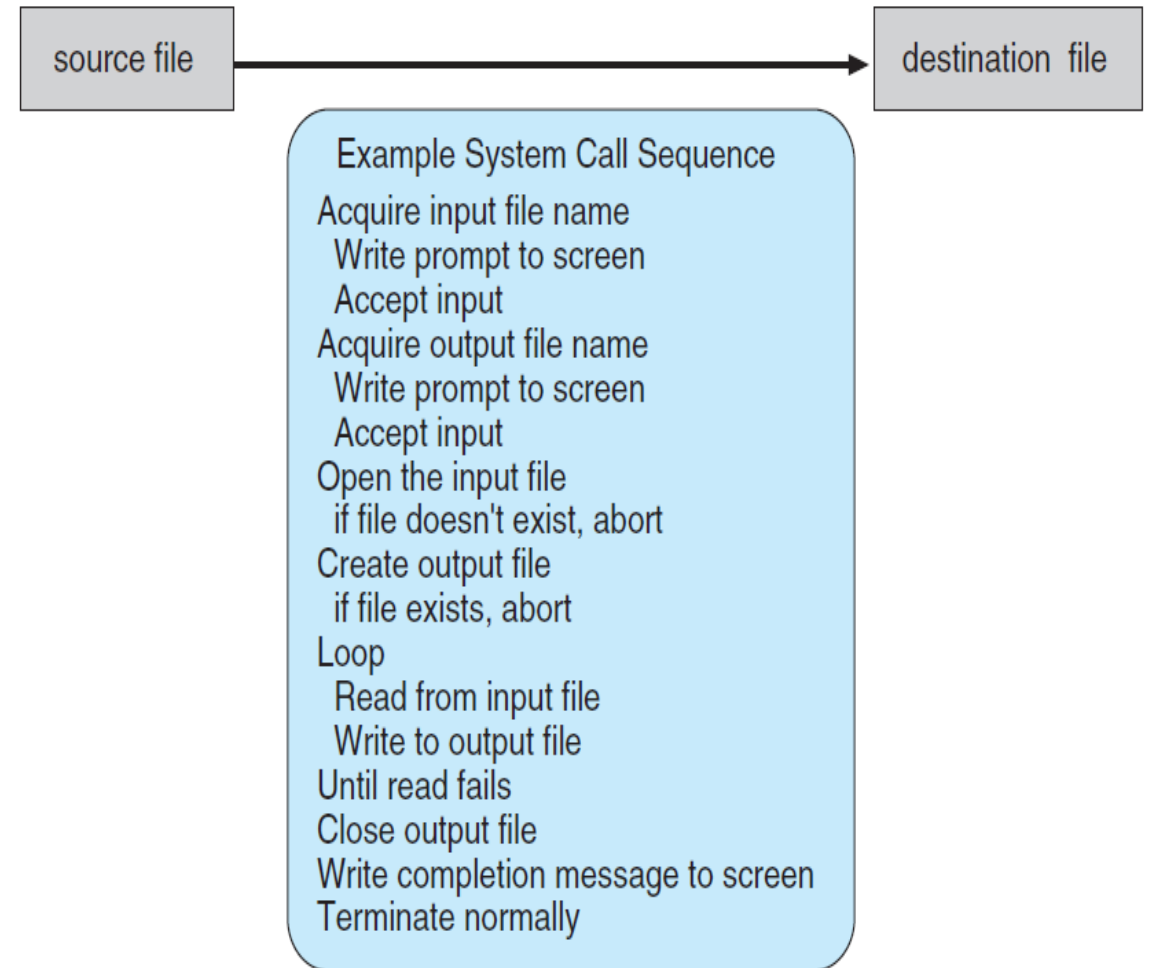


Figure: Example of how system calls are used

Structuring Operating system

1. Simple / Monolithic Structure

A Simple (Monolithic) Operating System Structure is an OS design in which the entire operating system runs as one large program in kernel mode.

All OS services are part of a single kernel and can directly communicate with one another.

In a monolithic kernel, components such as process management, memory management, file systems, device drivers, and networking are all integrated into one large kernel.

Example – MS DOS and the original UNIX operating system

Structuring Operating system

1. Simple / Monolithic Structure

How it works?

- A user program requests a service using a system call.
- The request enters the kernel and the appropriate kernel module performs the requested operation.
- Since all modules are in the same address space, they can directly call each other.
- The result is returned to the user program.

Structuring Operating system

1. Simple / Monolithic Structure

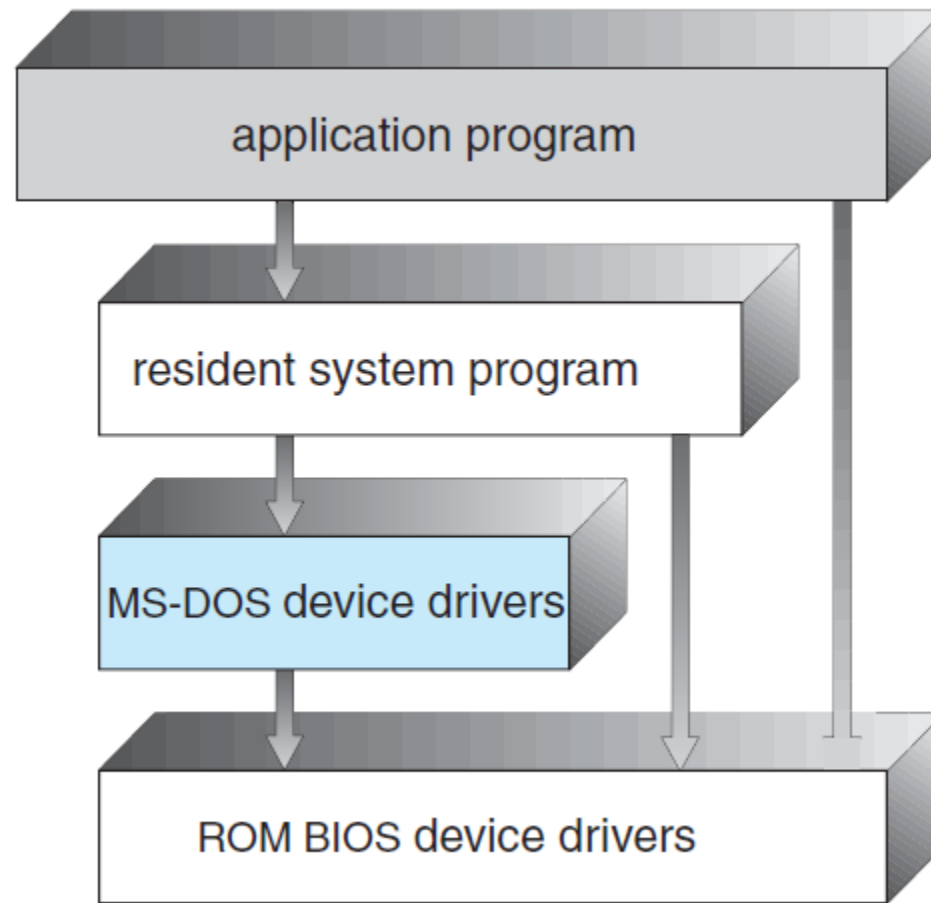


Figure: MS-DOS layer structure

Structuring Operating system

1. Simple / Monolithic Structure

All code runs in kernel space, with no separation between kernel and user-level processes.

The kernel is designed to provide all operating system services, including memory management, process scheduling, device drivers, and file systems, in a single, large binary file.

Advantages:

- Simple to implement when functionalities are limited
- Provides high performance since system calls can be made directly to the kernel without the overhead
- Efficient communication

Structuring Operating system

1. Simple / Monolithic Structure

Dis-advantages:

- Less secure and stable
- Lack of hardware support
- Since all code runs in kernel space, any vulnerabilities or bugs in the kernel can potentially affect the entire system.
- If a user-level process crashes, it can bring down the entire system since there is no separation between kernel and user-level processes.

Structuring Operating system

2. Layered Structure

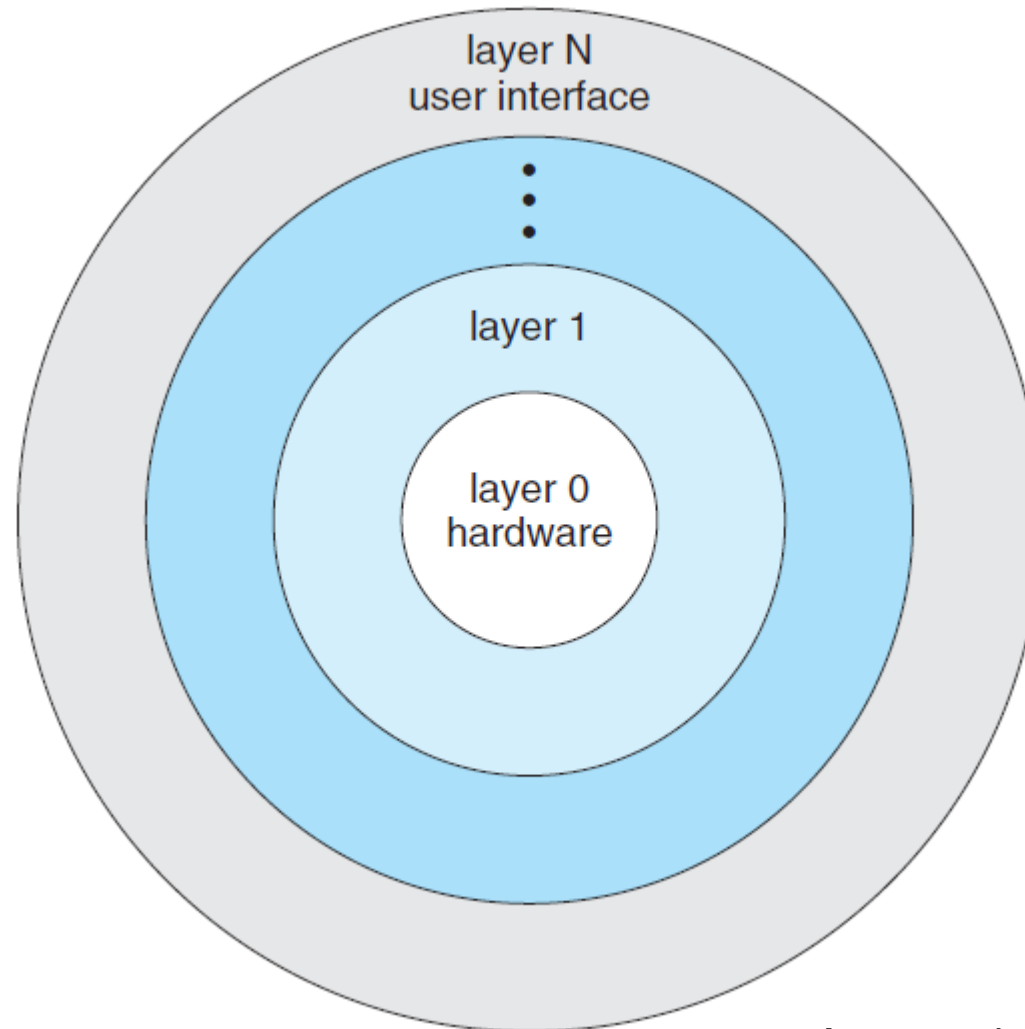


Figure: A layered operating system

Structuring Operating system

2. Layered Structure

In layered approach, the operating system was broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.

The different services of the operating system are split into various layers.

Each layer is responsible for performing a specific function and can only access the services provided by the layers below it.

For example: *layer M—consists of data structures and a set of routines that can be invoked by higher-level layers. Layer M, in turn, can invoke operations on lower-level layers.*

Structuring Operating system

2. Layered Structure

Each layer is implemented only with operations provided by lower-level layers.

A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

Hence, each layer *hides the existence of certain data structures, operations, and hardware from higher-level layers.*

Example: Early versions of **THE** Operating System

Structuring Operating system

2. Layered Structure

Advantages:

- *Changes in one component affect only that component, and no others.*
- *Simplicity of construction and debugging, making it easier to develop, test, and maintain.*

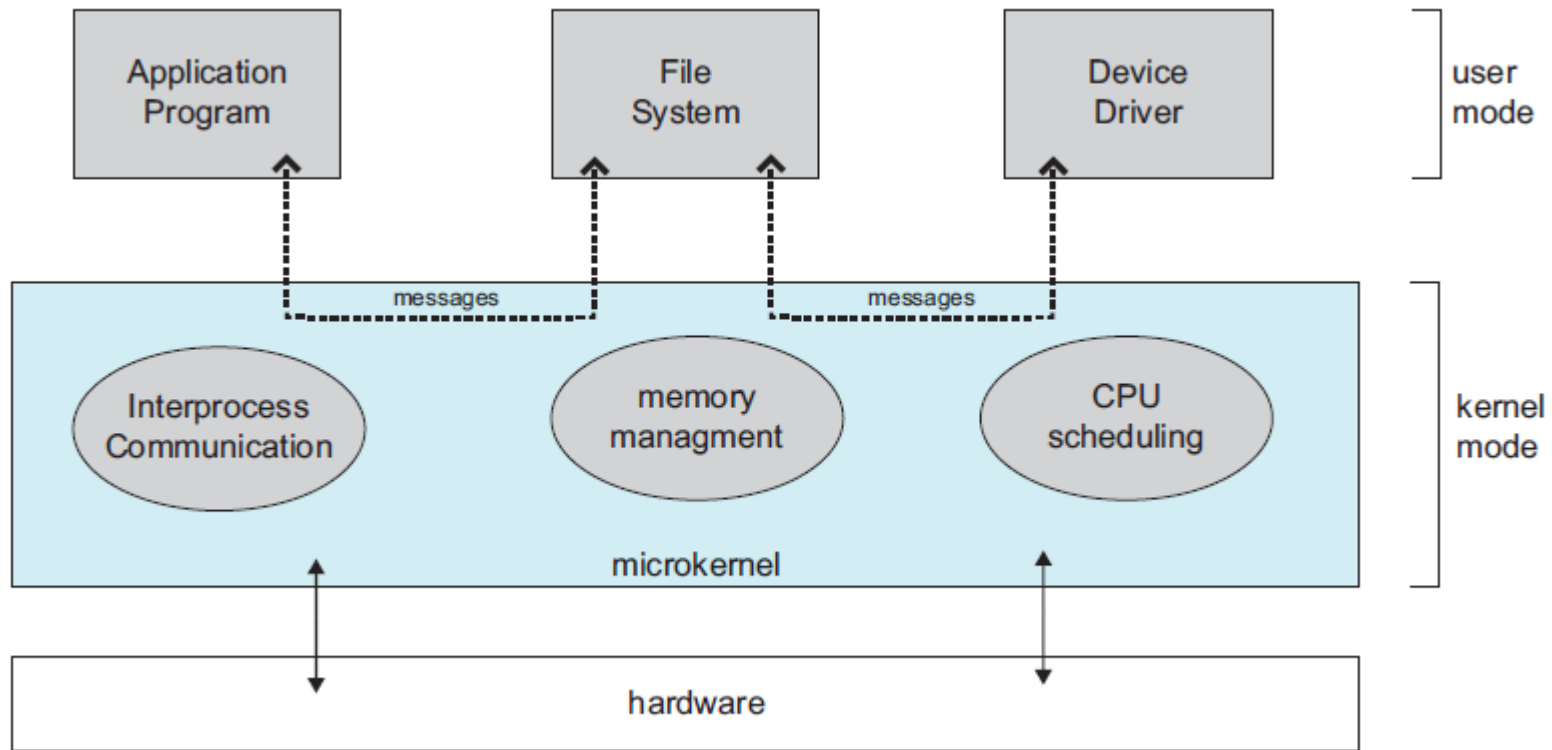
Dis-advantages:

- *The challenges of appropriately defining the functionality of each layer.*
- *The overall performance of such systems is poor due to the **overhead** of requiring a user program to traverse through multiple layers to obtain an operating-system service.*

Recommendation: *Fewer layers with more functionality are being designed*

Structuring Operating system

3. Microkernels Structure



Structuring Operating system

3. Microkernels Structure

In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called **Mach** that modularized the kernel using the microkernel approach.

This method removes all nonessential components from the kernel and implementing them as system and user-level programs. Thus the kernel is smaller now.

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space.

Services are provided through **message passing communication**

Structuring Operating system

3. Microkernels Structure

Advantages

- It makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.
- The resulting operating system is easier to port from one hardware design to another.
- Provides more security and reliability, since most services are running as user

Dis-advantages

- The performance of microkernels can suffer due to increased system-function overhead
- Operating system may have to switch from one process to the next to exchange the messages. The overhead involved in copying messages and switching between processes has been the largest impediment to the growth of microkernel-based operating systems.

Structuring Operating system

4. Modules / Loadable kernel modules

The kernel has a set of core components and can link in additional services via modules, either at boot time or during run time.

A **Loadable Kernel Module (LKM)** is an object file that contains code to extend the running kernel, or so-called base kernel, of an operating system.

Used to add support for new hardware (as device drivers), file systems, or system calls.

The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running without rebooting the system.

Structuring Operating system

4. Modules / Loadable kernel modules

It uses object-oriented approach

When the functionality provided by an LKM is no longer required, it can be unloaded in order to free memory and other resources.

Advantages

- Allows for greater flexibility and modularity in the design of an operating system
- Functionality can be loaded and unloaded as and when required
- It doesn't require recompiling the kernel every time a change was made
- Modules do not need to invoke message passing in order to communicate

Example: Solaris, Linux, Mac OS X, Windows

Structuring Operating system

4. Modules / Loadable kernel modules

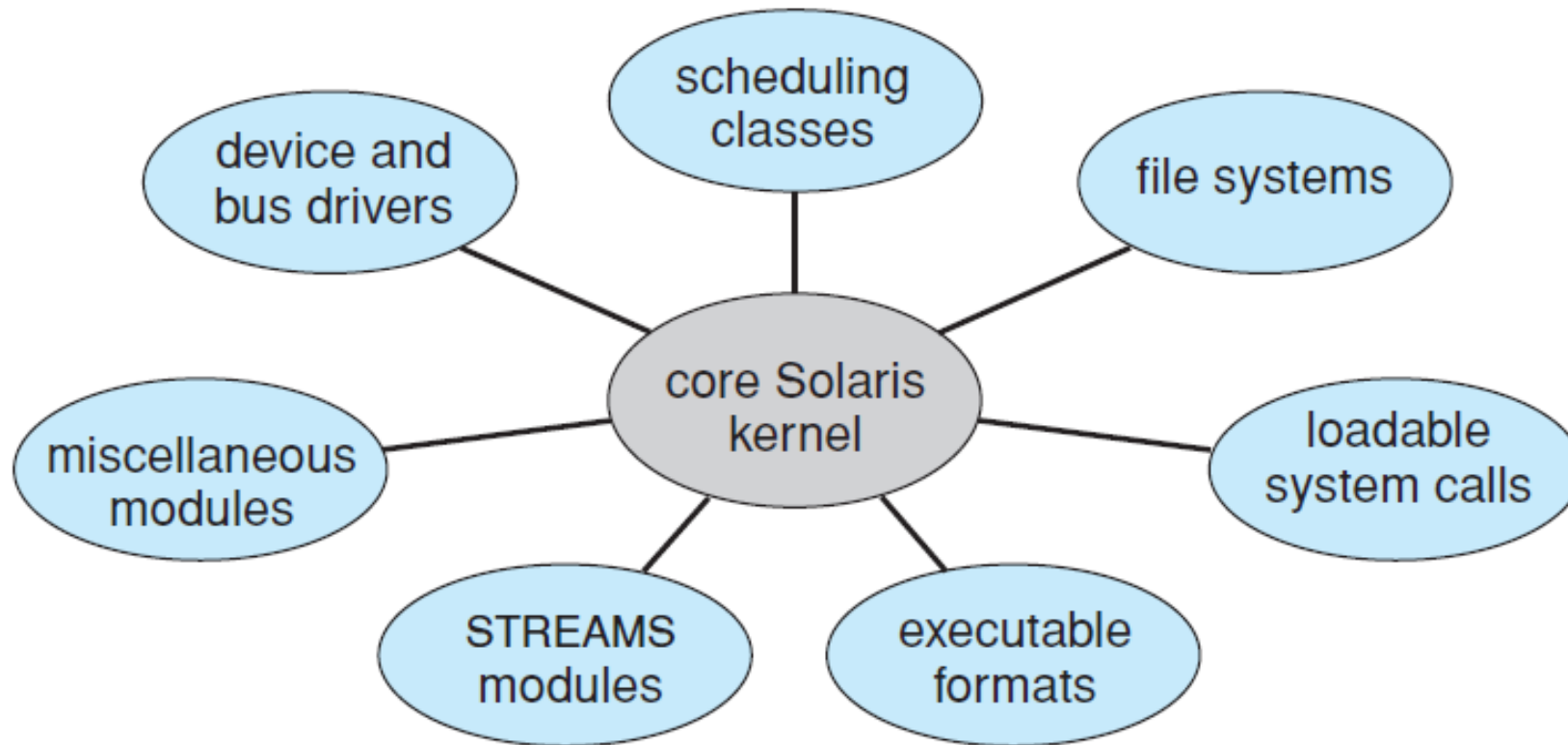


Figure: Solaris loadable modules.

Structuring Operating system

5. Hybrid Systems

*Most modern operating systems are actually **not one pure model***

- *Hybrid combines multiple approaches to address performance, security, usability needs*
- ***Linux and Solaris kernels** in kernel address space, so monolithic, plus modular for dynamic loading of functionality*
- ***Windows** mostly monolithic, plus microkernel for different subsystem personalities*

Structuring Operating system

5.1 macOS and iOS

Apple's macOS operating system is designed to run primarily on desktop and laptop computer systems, whereas iOS is a mobile operating system designed for the iPhone smartphone and iPad tablet computer.

The general architecture of these two systems has various layers include the following:

- 1. **User experience layer** – Aqua vs Springboard*
- 2. **Application framework layer** – Cocoa vs Cocoa touch*
- 3. **Core frameworks** - support graphics and media including, Quicktime and OpenGL.*
- 4. **Kernel** – Darwin (Mach microkernel and BSD UNIX kernel)*

Structuring Operating system

5.1 macOS and iOS

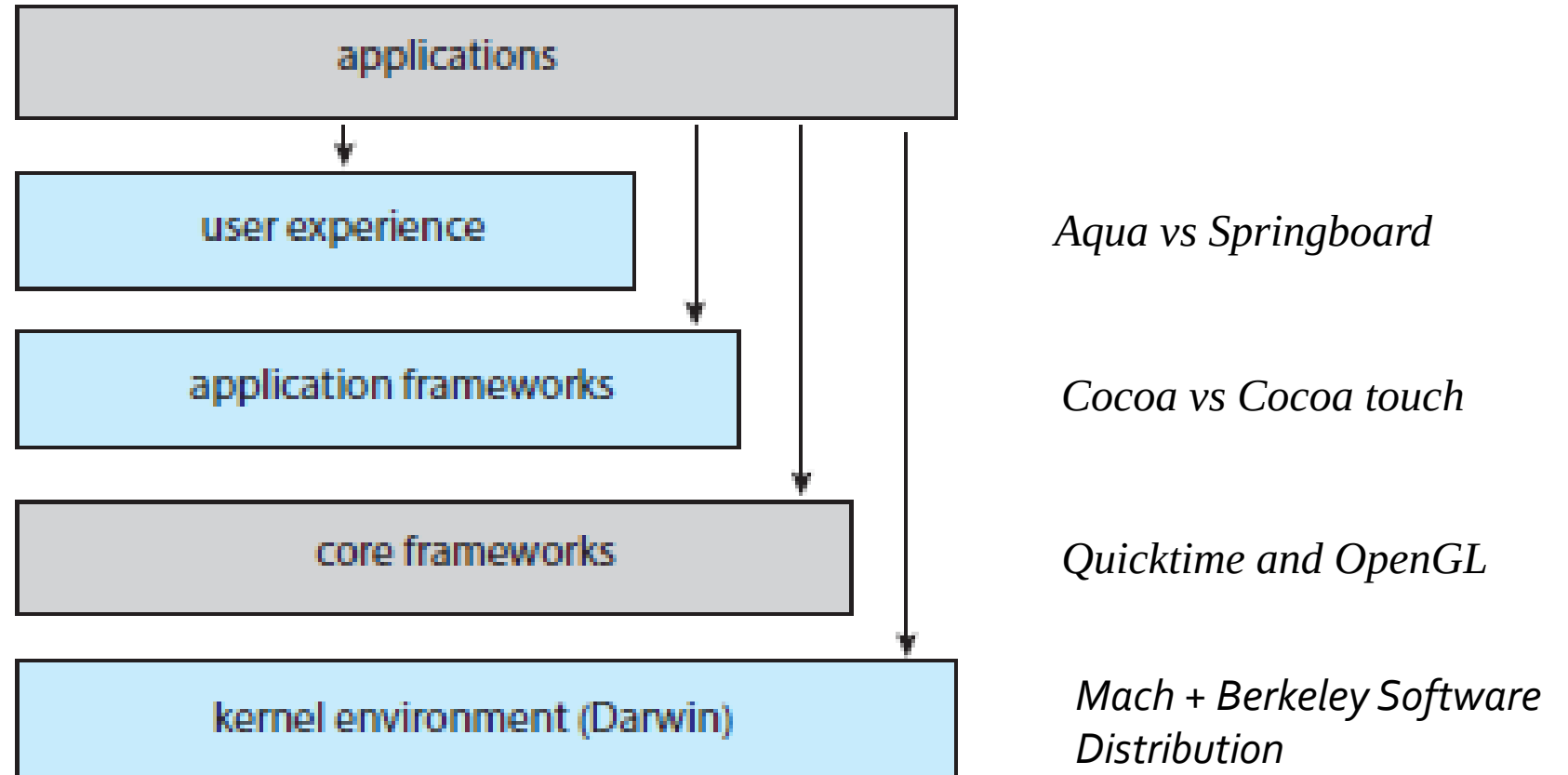


Figure: Architecture of Apple's macOS and iOS operating systems

- it combines ideas from layered architecture, microkernel architecture, and modular kernel architecture.

Structuring Operating system

5.1 macOS and iOS

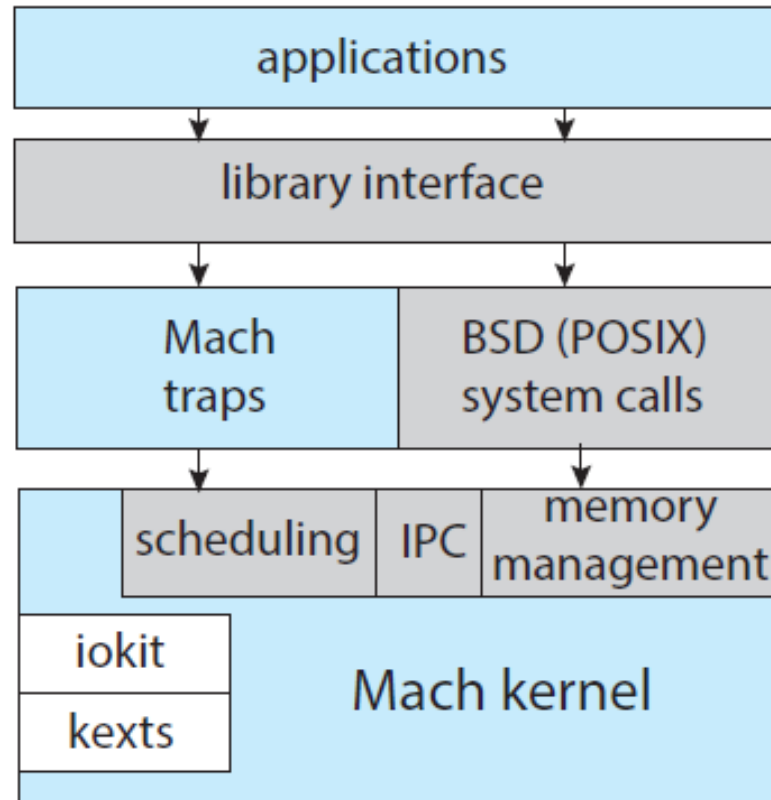


Figure: The structure of Darwin

Structuring Operating system

5.1 macOS and iOS

User experience layer defines the software interface that allows users to interact with the computing devices.

Application frameworks layer includes the Cocoa and Cocoa Touch frameworks, which provide an API for the Objective-C and Swift programming languages.

Core frameworks layer defines frameworks that support graphics and media including, Quicktime and OpenGL.

Kernel environment also known as Darwin which includes the Mach microkernel and the BSD UNIX kernel.

Structuring Operating system

5.1 macOS and iOS

Some significant distinctions between macOS and iOS include the following:

*Because macOS is intended for desktop and laptop computer systems, it is compiled to run on **Intel architectures**. iOS is designed for mobile devices and thus is compiled for **ARM-based architectures**.*

*The iOS operating system is generally much **more restricted** to developers than macOS. For example, iOS **restricts access to POSIX and BSD APIs** on iOS, whereas they are openly available to developers on macOS.*

Structuring Operating system

5.1 macOS and iOS

Darwin, which uses a hybrid structure. It is a layered system that consists of the Mach *microkernel and the BSD UNIX kernel*.

Darwin provides two system-call interfaces: Mach system calls and BSD system calls.

These system calls has a rich set of libraries to provide networking, security, and programming language support.

Mach provides fundamental operating system services, including memory management, CPU scheduling, and interprocess communication

Summary

Structure	Advantage	Disadvantage	Example
Monolithic	High performance	Difficult to maintain and debug	Linux, early Unix
Microkernel	Better modularity, security	Potential performance overhead	QNX, MINIX, L4
Layered	Clear separation of concerns	Performance overhead, strict layer boundaries	THE, early Unix
Loadable Modules Kernel	Dynamic loading/unloading, flexibility	Security risks, complexity	Linux, Windows
Hybrid	Performance, modularity	Complexity, potential trade-offs	Windows NT, macOS (XNU)

References

1. **Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.**

BCSE303L

OPERATING SYSTEMS

Module 1 Lecture 3

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

1. Start the design by defining goals and specifications
2. Choice of hardware
3. System types
4. Specify the requirements
 - *User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast*
 - *System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient*
5. Separation of policy from mechanism

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

5. Separation of policy from mechanism

- *Mechanisms determine how to do something, policies decide what will be done.*
- *The separation of policy and mechanism is important for flexibility*
- *Policies are likely to change across places or over time, and each change in policy would require a change in the underlying mechanism*
- *Microkernel-based operating systems allows more advanced mechanisms and policies to be added via user-created kernel modules or user programs.*
- *Policy decisions are important for all resource allocation*

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

6. Implementation

- *Early operating systems were written in assembly language. Now, most are written in higher-level languages such as C or C++ with small amounts of the system written in assembly language.*
- *For example: Android kernel is written in C with some assembly language code; system libraries are written in C or C++, and its application frameworks in Java*
- *The **advantages** of using a higher-level language the code can be written faster, more compact, easier to understand and debug, and easier to port to other hardware.*
- ***Disadvantages:** reduced speed and increased storage requirements*

Abstractions

Abstractions are a way to represent complex systems or components in a simplified way, exposing only essential features and hiding implementation details.

Using abstractions, Operating System provides a more comprehensible and manageable interface for users and developers, while hiding the intricate details of the underlying system.

Some example abstractions are:

Process Abstraction: Represents a process as a single entity, hiding details like memory layout, registers, and execution state.

File Abstraction: Represents files as a sequence of bytes, hiding storage details like disk blocks and inode structures (metadata).

Socket Abstraction: Represents network connections as a socket, hiding details like protocol specifics and network hardware.

Device Abstraction: Represents hardware devices as a standardized interface, hiding details like device registers and interrupts.

Abstractions

Abstraction provides,

- ❑ *Simplification: Hide complexity, making it easier to understand and interact with the system.*
- ❑ *Modularity: Enable separate development and maintenance of components.*
- ❑ *Portability: Make it easier to move code between different platforms or architectures.*
- ❑ *Security: Limit access to sensitive information, improving security*

Abstraction can be achieved through,

- ❑ *Encapsulation: Hiding implementation details within a module or component.*
- ❑ *Interfaces: Providing a standardized way to interact with a component or system.*
- ❑ *APIs: Offering a set of functions or services to access a system or component.*

Processes in Operating System

A process in an operating system is essentially a program in action/execution.

- *It's the active version of a program that's currently running instructions.*
- *A process is the fundamental unit of work for the operating system.*

Some Process Types –

1. *User Processes:* Execute user programs.
2. *System Processes:* Perform OS functions, like process management.
3. *Daemon Processes:* Run in the background, providing services.

Resources in Operating System

- *It refers to the components or facilities that are managed and allocated to processes or programs to enable them to execute and complete their tasks.*
- *Essential components that programs need to run.*
- *The OS acts as resource manager, allocating these resources fairly and efficiently among all the running programs.*
- *The Operating System manages resources by doing the following: allocation, deallocation, scheduling, protection and sharing.*

Resources in Operating System - categories

▪ Physical Resources

- CPU (Central Processing Unit)
- Memory (Main Memory or RAM)
- I/O Devices (Keyboard, Mouse, Printer, etc.)
- Storage Devices (Hard Disk, Solid State Drive, etc.)

▪ Virtual Resources

- Virtual Memory (Swap Space or Page File)
- Virtual CPUs (in a virtualized environment)

▪ Software Resources

- Files and Directories
- I/O Streams (Input/Output Channels)
- Sockets (Network Connections)
- Semaphores (Synchronization Mechanisms)

▪ Abstract Resources

- Processes (Threads, Tasks, or Jobs)
- Threads (Lightweight Processes)
- Synchronization Objects (Mutexes, Locks, etc.)

Security in Operating System

Security in an Operating System (OS) is crucial to protect the system, its resources, and user data from unauthorized access, malicious activities, and potential threats.

Why Security?

- ☐ *Protect user data and privacy*
- ☐ *Prevent unauthorized system access*
- ☐ *Ensure the integrity of data and programs*
- ☐ *Maintain system availability and reliability*
- ☐ *Comply with security regulations and standards*
- ☐ *Support secure communication and transactions*
- ☐ *Provide a trusted platform for applications and services*

Security in Operating System

Some of the security design issues are:

1. **Access Control:** The OS manages access to resources, such as files, directories, and devices, through mechanisms like *permissions, access control lists (ACLs), and attribute-based access control (ABAC)*.
2. **Authentication:** The OS verifies the identity of users and processes through authentication mechanisms like *passwords, biometrics, and Kerberos*.
3. **Authorization:** The OS determines what actions a user or process can perform on a resource, based on their *role, privileges, and permissions*.
4. **Encryption:** The OS provides encryption mechanisms, like *file encryption and secure sockets layer (SSL)*, to protect data from unauthorized access.
5. **Integrity:** The OS ensures the *integrity of data and programs* by preventing unauthorized modifications or tampering.

Security in Operating System

Some of the design issues are:

6. Confidentiality: The OS protects sensitive information from unauthorized access, use, or disclosure.
7. Availability: The OS ensures that resources and services are available to authorized users when needed.
8. Threat Protection: The OS provides protection against various threats, such as *malware, viruses, Trojan horses, spyware, and ransomware*.
9. Vulnerability Management: The OS patch management and vulnerability scanning help identify and remediate vulnerabilities.
10. Auditing and Logging: The OS provides auditing and logging mechanisms to track security-related events and monitor system activity.

Networking in Operating System

- Networking refers to the ability of an Operating System to connect to and communicate with other devices, systems, or networks.
 - **Resource Sharing:** Sharing files, printers, and other resources between devices.
 - **Communication:** Exchanging data, messages, or requests between devices or systems.
 - **Remote Access:** Accessing remote systems or networks, like VPNs or remote desktops.

Multimedia in Operating System

Multimedia refers to the ability of an Operating System to handle and manage various forms of media, such as:

- **Audio:** Playing, recording, and manipulating audio files.
- **Video:** Playing, recording, and manipulating video files.
- **Graphics:** Rendering and manipulating graphical content.
- **Images:** Handling and manipulating image files.

Multimedia in Operating System

This involves

- **Device Drivers:** Managing device drivers for multimedia hardware, like sound cards or graphics cards.
- **Media Formats:** Supporting various media formats, like MP3, AVI, or JPEG.
- **Media Players:** Providing media player applications or frameworks.
- **Multimedia APIs:** Offering APIs for developers to access multimedia functionality.

Multimedia in Operating System

- Multimedia data includes continuous media in the form of audio or video files and conventional files.
- Multimedia *demands a very high data rate* and real-time delivery of the data. For any video display, *a rate of 24 to 30 frames per second* is necessary for the video to appear smooth to the human eye.
- The Multimedia API is a *collection of low-level APIs* that support flexible application development.
- Multimedia API can provide following applications: Video encode and decode, Video convert, Image and video processing, Object detection and classification, etc.

References

1. **Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.**